

you view the contents of the registers, the memory and I/O port values, execute one instruction at a time, and also write code in mnemonics instead of hand-assembled code.

While I was pursuing my undergraduate course, I began to look for a FOSS-based 8085 simulator. Simulators supplied with text books, and others available on the Internet either did not fulfil my needs, or were not FOSS. Then I recalled that *LFY* had once published a review of GNUSim8085, and downloaded the software. The basic features listed on the official site are:

- It comes with an 8085 assembler.
- The editor has syntax highlighting.
- *Register* and *Flags* display are available.
- Has a hex-to-decimal converter.
- Provides a view of memory, I/O and stack contents.
- Step-by-step execution support.
- Print support.
- Has UI in different languages.

Now, let's take a detailed look at this software.

The interface

When the software is launched, the editor starts with a template—sample code you can edit—or, you can open a new file, and start coding. The editor lines are numbered. The code highlighting is good, and makes it easier to read the code. To run or assemble code, you need to save it first, before compilation is allowed.

On the left-hand pane, all the registers and flags are listed in a very easy-to-read manner. All the register values are displayed in hexadecimal format. The hex-to-decimal conversion box, which is very useful in such applications, is in this pane. Personally, I feel that the implementation ought to have been a bit different. To convert from decimal to hex, or vice-versa, you need to type in the number and click the correct button to make the conversion. I would have liked it if it automatically converted as the input box value change; this would make things much faster and more convenient.

Then the I/O Ports space is displayed as a spin-box. The spin-box index is the port number, and the text box beside it shows the contents of this I/O port. Similarly, the *Memory* space is shown with another spin-box. Note that the values shown in both are all in decimal. The default input for these is in decimal, but to enter a value in hex, you add an 'h' after it; if the number starts with a letter of the alphabet, you need to add a zero before it. For example, AB is entered as '0abh'.

On the right-hand side, there are three tabs. The *Data* tab shows the data variables you have defined with the assembler EQU pseudo op-code (discussed later). The next tab shows the stack state. After you initialise the stack pointer (SP) and execute *PUSH*, *CALL* or other stack-manipulation instructions, this tab will show the address and value of the entire stack, along with the associated pushed register pair or program label (for call). This is especially helpful when

you're debugging code with many nested subroutines, or using stack-manipulation instructions like *XTHL*. When pushing the contents of the PSW register (Accumulator and Flags), the Register name under the "*Proc/Reg*" field is incorrect, although the performed operations have no problem. This is a really helpful tool for debugging.

The third tab contains a virtual keypad, consisting of keys corresponding to each instruction. It looks good, but I do not think this is a feature that will get used a lot. It is much easier and simpler to type in mnemonics, than to click buttons to code.

In the text box on the top labelled "*Load me at*", you can enter an address location, and make the assembled code load, starting at that memory location. The default input is decimal; to enter hex, use the same rules as before. An empty or invalid address means the program will be loaded at 4200h by default.

The '*Assembler*' menu has three items. The first, '*Assemble (F8)*', is used to assemble the program and load it into memory (at the load address). There is no practical use of only assembling the code in the current version, as you cannot generate an output assembled binary file that you could burn into the memory interfaced with 8085. The next, '*Execute (F9)*', would execute the compiled program. The following entry is '*Show Listing (Ctrl + L)*', which shows you the assembled machine code, along with the memory address op-codes, mnemonics and comments. This window is especially useful when you have successfully run a program in the simulator, and are ready to run it on the actual 8085; you simply need to enter the proper load location, and then either display the listing, or save it into a file and print it out, before entering it into the actual machine. This saves you from the hassles of hand assembly.

The '*Debug*' menu has the '*Step in (F5)*', '*Step Over (F6)*' and '*Step Out (F7)*' options, along with the program breakpoint management. These features are very helpful for debugging code by inspecting the register and memory contents at each step, or at breakpoints.

From the '*Reset*' menu, you can reset the state of registers, I/O ports, flags and memory, individually or all at once. It is useful when you want to remove old data from the simulator without writing extra 8085 code.

The toolbar is concise and clutter-free. It contains the normal save, load and new icons, apart from execution and debug short-cut buttons. The icon of the big red cross is to stop (terminate) execution when you are stepping through code, or in between.

At the bottom of the window is the assembler error message output panel, where you see any errors, with the line number and a brief description of the error. Clicking the error message highlights the line with the error.

Assembler

This simulator comes with an assembler: you write the code in Intel 8085 mnemonics, and it is converted to machine code